OneCare Systems

Sprint 5 Retrospective

2026/05/27 – 2026/06/17

2026/06/18

Attendees

- Vipusa
- Thushajini
- Sweeskaran
- Kenuja

Went Well

1. Spring Boot backend architecture scaffold was successfully completed and established as the foundation for backend development.
2. Testing frameworks were installed and configured, enabling future test implementation.
3. Initial GitHub Actions CI pipeline setup was started, and repository workflows were established.
4. Team responsibilities and issue ownership were clearly defined at the beginning of the sprint.
5. Core project setup activities were completed, providing a foundation for Sprint 6 development.
6. MySQL schema, JPA entities, and the Login API with JWT authentication were completed by the sprint close, after resolving earlier environment blockers.

To Improve

1. Foundational tasks such as Docker Compose setup and CI pipeline completion were not completed early enough in the sprint.
2. Several planned tasks were blocked due to dependencies on unfinished foundational work, most critically, Docker Compose (DDP-#06).
3. Sprint planning underestimated the impact of task dependencies and task sequencing.
4. Environment setup issues caused delays in database development activities.
5. Team members required additional time to learn and configure unfamiliar technologies such as GitHub Actions CI/CD.
6. Technical blockers were not escalated early enough for team assistance.
7. Sprint planning did not adequately account for learning curves and setup-related risks.
8. Progress reviews should be conducted more frequently to identify and address blockers earlier.

9. Dependency-heavy tasks should be prioritized before assigning dependent feature development work.
10. Two tickets deviated from their written specification without documentation Jest was replaced with Vitest (DDP-#01), and BCrypt was used instead of the required Argon2id with lockout (DDP-#15). Deviations from an approved ticket or SDS element must be recorded as an ADR rather than implemented silently.
11. Tickets should be checked line-by-line against their acceptance criteria before being marked complete DDP-#09, #10, and #13 were closed while still listing an incomplete dependency (DDP-#06), which was not caught until a later audit.

Action Items

Action	Owner	Due
Finish Docker Compose setup — Spring Boot, React, MySQL 8, Redis containers + health checks (DDP-#06)	Vipusa	Sprint 6 – Week 1
Add deploy-to-staging stage, artifact upload verification, README CI badge to CI pipeline (DDP-#05)	Vipusa	Sprint 6 – Week 1
Complete React integration: Axios instance, JWT interceptor, auth context, login UI, dashboard UI (DDP-#08)	Vipusa	Sprint 6 – Week 2
Confirm Mockito + React Testing Library setup; document Jest→Vitest decision as an ADR (DDP-#01)	Thushajini	Sprint 6 – Week 1
Wire coverage artifacts, coverage badge, and CI summary into the pipeline (DDP-#02)	Thushajini	Sprint 6 – Week 1
Write test plan document (DDP-#03)	Thushajini	Sprint 6 – Week 2
Complete remaining 2 of 3 required smoke tests (DDP-#04)	Thushajini	Sprint 6 – Week 2
Write seed data file and environment profile configs (DDP-#11, DDP-#12)	Sweeskaran	Sprint 6 – Week 1
Re-verify DDP-#09/#10/#13 against the real containerized environment once DDP-#06 lands	Sweeskaran, Kenuja	Sprint 6 – Week 1
Audit RBAC guards, permission matrix, and role-redirect against the original AC line-by-line (DDP-#14)	Kenuja	Sprint 6 – Week 1
Decide: implement Argon2id + account lockout as specified, or raise an ADR to keep BCrypt with supervisor sign-off (DDP-#15)	Kenuja	Sprint 6 – Week 1
Implement password reset flow (DDP-#16)	Kenuja	Sprint 6 – Week 2
Review task dependencies during sprint planning; sequence critical-path tasks first	All Members	Next Sprint Planning

Metrics

- Issues planned: 16
- Issues completed: 4
- Issues partially completed: 5
- Issues not completed: 7
- Sprint completion rate: 25%
- Velocity: Baseline sprint (story point velocity to be established from Sprint 6)
- CI pipeline pass rate: Partial — lint, build, unit test, and security scan stages pass; deploy-to-staging stage not yet configured
- Test coverage: Coverage tooling runs locally (JaCoCo/Vitest) but is not yet published through CI

Completed Work

Issue ID	Task	Status
DDP-#07	Spring Boot backend scaffold	Completed
DDP-#09	MySQL schema implementation	Completed
DDP-#10	JPA entities (Patient, User, Appointment)	Completed
DDP-#13	Login API and JWT authentication	Completed
DDP-#01	Test framework installation (JUnit/Mockito/Jest)	Partially Completed
DDP-#02	Coverage reporting in CI	Partially Completed
DDP-#05	GitHub Actions CI pipeline	Partially Completed
DDP-#08	React application scaffold	Partially Completed
DDP-#14	RBAC implementation	Partially Completed

Unfinished Work and Reasons

Issue ID	Task	Reason
DDP-#01	Confirm Mockito / RTL setup	Jest was substituted with Vitest without a documented ADR; Mockito and React Testing Library configuration not confirmed
DDP-#02	Coverage reporting in CI	JaCoCo/Vitest coverage runs locally but is not exported as CI artifacts, no coverage badge, no CI summary
DDP-#03	Write test plan document	Not started during sprint
DDP-#04	Write smoke tests	Only 1 of 3 required smoke tests exists (JWT integration test only)

DDP-#05	Deploy-to-staging CI stage	No containerised target environment exists yet — direct consequence of DDP-#06 not being completed
DDP-#06	Docker Compose setup	Not started during the sprint; root cause not identified — became the sprint's critical-path blocker
DDP-#08	React scaffold — integration work	Routing and layout completed, but Axios config, JWT interceptor, auth context, login, and dashboard implementation depended on DDP-#13 landing first
DDP-#11	Create seed data	Deprioritised (P2); also blocked on Docker Compose (DDP-#06)
DDP-#12	Configure profiles and properties	Blocked by incomplete Docker Compose setup (DDP-#06)
DDP-#14	RBAC — full verification	Role system exists, but @PreAuthorize guards, permission matrix, and role-redirect were not verified against the ticket's acceptance criteria
DDP-#15	Argon2id hashing and account lockout	Ticket specified Argon2id and lockout after 5 failed attempts; implementation uses BCrypt with neither Argon2id nor lockout, and the deviation was not documented
DDP-#16	Password reset flow	Correctly blocked — depends on DDP-#13 (done) and DDP-#15 (not done)

Root Cause Analysis

The primary reason Sprint 5 objectives were not fully achieved was the delay in completing foundational infrastructure tasks. The MySQL schema implementation (DDP-#09) was initially delayed due to SQL Workbench connection issues encountered in the development environment, but was resolved and completed by the end of the sprint, along with the dependent JPA entity work (DDP-#10) and the Login API (DDP-#13).

Docker Compose setup (DDP-#06), however, was never started during the sprint and became the sprint's critical-path blocker: it sits upstream of the CI pipeline's staging-deploy stage (DDP-#05), the seed data and environment profile tickets (DDP-#11, DDP-#12), and is listed as a dependency for DDP-#09, #10, and #13 even though those three were marked complete. This raises an open question — to be confirmed in the Sprint 6 review — of whether that

database and authentication work was ever validated against the actual containerised target environment described in the SDS, or only against ad-hoc local setups.

The GitHub Actions CI pipeline (DDP-#05) progressed further than in the prior snapshot — lint, build, unit test, and security scan stages are now working — but the deploy-to-staging stage, artifact verification, and README CI badge remain outstanding, consistent with no containerised staging target existing yet.

Two tickets were also closed against a different specification than what was written without documenting the change: DDP-#01 substituted Vitest for the specified Jest, and DDP-#15 implemented BCrypt hashing instead of the specified Argon2id with account lockout. Neither deviation was recorded as an Architectural Decision Record, which the course guideline requires whenever implementation departs from an approved ticket or design element.

Despite these gaps, the sprint did establish the project's core backend, database, and authentication foundation, and produced concrete, ticket-level insight into dependency management, sprint estimation, and the need to verify acceptance criteria before closing an issue — all of which should directly improve execution in Sprint 6.

Lessons Learned

1. Complete foundational infrastructure and database tasks before starting dependent feature development.
2. Review and map task dependencies during sprint planning sessions.
3. Prioritize critical-path tasks early in the sprint — Docker Compose should have been a Day 1 task, not left to the end.
4. Monitor blockers continuously and escalate them sooner.
5. Establish realistic sprint commitments based on dependency analysis and team capacity.
6. Allocate buffer time for learning new technologies such as CI/CD platforms and development tools.
7. Resolve development environment issues immediately and seek team support when blockers persist.
8. Consider technical risk and learning effort during sprint estimation.
9. Document any deviation from a ticket's specification or the approved SDS as an ADR at the time it happens, rather than leaving it to be discovered in a later audit.

10. Verify a ticket against its full acceptance criteria — including its listed dependencies — before marking it Done on the board.

Sprint 6 Focus

1. Complete Docker Compose setup and re-verify the database and authentication work (DDP-#09, #10, #13) against the real containerised environment.
2. Finalise the CI/CD pipeline's deploy-to-staging stage and testing infrastructure (coverage artefacts, badge, CI summary).
3. Complete remaining environment and deployment setup tasks (seed data, environment profiles).
4. Resolve the two undocumented spec deviations — confirm or formally document the Vitest and BCrypt decisions — and complete RBAC verification and password reset.
5. Improve sprint planning through dependency analysis, risk assessment, and workload balancing.